NTNU

Quickstart Guide

Robotic Plant Acquisition with JetCobot & ROS 2

Document: JetCobot ROS 2 Operations & Acquisition Guide
Prepared by: Aymen Turki
Prepared for: Melissa & Erland

Hi Melissa and Erland,

This guide provides the practical setup required to run the JetCobot plant-acquisition pipeline, starting from simulation and progressing to the physical robot. For the complete project documentation, source code, launch files, and implementation details, please also refer to the project GitHub repository:

Robotic-Plant-Phenotyping

1 System Architecture

7-axis JetCobot with Jetson Orin Nano and wrist-mounted RGB camera.

The system is organized into four main software layers:

Plant Acquisition / Custom ROS 2 Nodes
⇓
MoveIt2
⇓
ROS 2 Jazzy
⇓
Ubuntu / Linux
⇓
PC or Jetson Orin Nano

1.1 Jetson Orin Nano

The **NVIDIA Jetson Orin Nano** is the onboard computer of the JetCobot. It provides the CPU/GPU resources required for ROS 2, robot control, motion planning, and vision processing.

NVIDIA Jetson Orin Nano used as the JetCobot onboard computer.

1.2 ROS 2 Jazzy Jalisco

ROS 2 Jazzy Jalisco is the robotics middleware running on top of Ubuntu/Linux. It provides communication between the robot driver, MoveIt2, camera, acquisition nodes, and other components.

ROS 2 Jazzy Jalisco.

ROS 2 communicates mainly through:

- **Topics** — continuous data such as joint states, images, and TF.
- **Services** — request/response operations.
- **Actions** — long-running operations such as trajectory execution.

1.2.1 Development PC

The development PC should run Ubuntu 24.04 LTS. If Windows is already installed, Ubuntu can be installed using a **dual-boot** configuration.

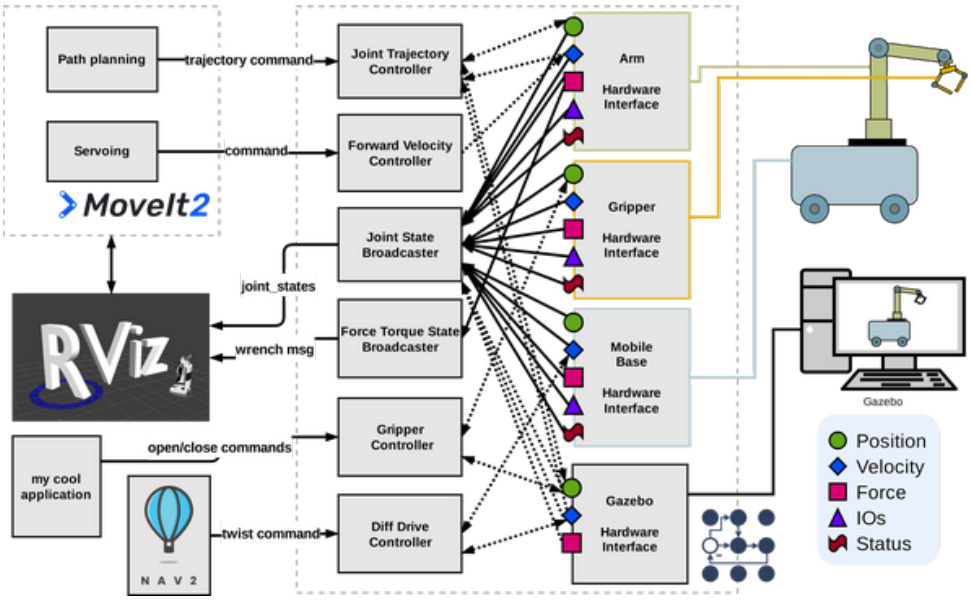
1.2.2 JetCobot

The JetCobot uses the Jetson Orin Nano as its onboard Linux computer. ROS 2 and the JetCobot software stack are installed on the Jetson for physical robot operation.

1.3 MoveIt2

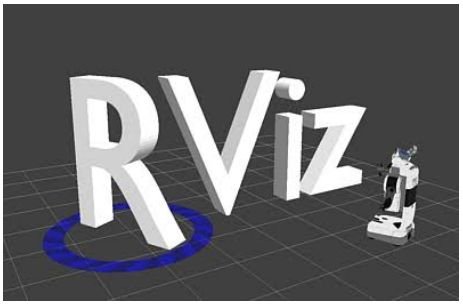
MoveIt2 provides inverse kinematics, trajectory planning, collision checking, and trajectory execution.

- **IK** — converts an end-effector pose into joint configurations.
- **Planning** — generates a feasible joint trajectory.
- **Collision checking** — verifies the planned motion.
- **Execution** — sends the trajectory to the robot controller.



1.4 RViz

RViz provides the 3D interface for visualizing the robot, planning poses, TF frames, and trajectories.



1.5 Gazebo

Gazebo provides the simulated robot and plant environment. It allows trajectories to be tested before applying motion to the physical JetCobot.

! SAFETY

Always validate new trajectories in Gazebo before executing them on the physical JetCobot.

2 Phase 1 — Simulation Setup

2.1 Step A: Install Ubuntu and ROS 2 Jazzy

Install Ubuntu 24.04 LTS on the development PC. A dual-boot installation can be used when Windows is already installed.

Install ROS 2 Jazzy and the development tools according to the official ROS 2 installation procedure.

```
sudo apt update
sudo apt upgrade -y

sudo apt install ros-jazzy-desktop -y
sudo apt install ros-dev-tools -y
```

Source ROS 2:

```
source /opt/ros/jazzy/setup.bash
```

Optionally add it to `.bashrc`:

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
```

2.2 Step B: Configure the Jetson

Configure the Jetson Orin Nano using the JetCobot system image and Yahboom instructions.

- Follow the Yahboom physical setup and first-power-on guide: *JetCobot Unboxing and Setup Guide*.
- Obtain the JetCobot system image and installation instructions from the shared Yahboom resource drive: *Yahboom JetCobot Resource Drive*.
- Configure the Jetson network and verify SSH access.

```
ssh jetson@<JETSON_IP>
```

! SAFETY

Use the system image intended for the JetCobot/Jetson configuration provided by Yahboom. Do not replace it with an arbitrary Jetson image.

2.3 Step C: Create the ROS 2 Workspace

Create the workspace on the development PC:

```
mkdir -p ~/jetcobot_ws/src
cd ~/jetcobot_ws/src
```

Place the JetCobot packages supplied by Yahboom and the project packages inside `src/`.

The main packages are:

- `jetcobot_description`
- `jetcobot_driver`
- `jetcobot_moveit`
- `jetcobot_gazebo`
- plant-acquisition packages

The official Yahboom repository contains the JetCobot Linux, ROS, robotic-arm, URDF, and kinematics resources.

> ⇒ **NOTE**
>
> Use the package versions contained in the project workspace and shared Yahboom resources. Do not mix unrelated JetCobot ROS packages from other repositories.

2.4 Step D: Install Dependencies and Build

```
cd ~/jetcobot_ws

source /opt/ros/jazzy/setup.bash

rosdep install --from-paths src --ignore-src -r -y

colcon build --symlink-install

source install/setup.bash
```

After building, verify that the packages are visible:

```
ros2 pkg list | grep jetcobot
```

2.5 Step E: Test MoveIt2 + RViz

Launch the MoveIt2/RViz configuration from this project:

```
source /opt/ros/jazzy/setup.bash
source ~/jetcobot_ws/install/setup.bash

ros2 launch jetcobot_moveit moveit_rviz.launch.py
```

Source:

`moveit_rviz.launch.py`

Verify:

- the JetCobot model is visible;
- the `arm` planning group is available;
- the end-effector marker can be moved;
- MoveIt2 can generate a trajectory.

2.6 Step F: Test MoveIt2 + Gazebo

Launch the complete simulated environment:

```
source /opt/ros/jazzy/setup.bash
source ~/jetcobot_ws/install/setup.bash

ros2 launch jetcobot_gazebo moveit_gazebo.launch.py
```

Source:

`moveit_gazebo.launch.py`

Check that the JetCobot and plant are correctly loaded.

2.7 Step G: Test the Acquisition Scan

Once the Gazebo environment is working, use the **scan launch file already provided in the project repository**.

Do not replace it with a manually constructed `ros2 run plant_acquisition scan_hemisphere_node` command.

First inspect the available launch files:

```
ros2 pkg prefix plant_acquisition
```

Then locate the package launch directory and use the project's **scan launch file**:

```
ros2 launch plant_acquisition <scan_launch_file>
```

The exact launch filename should be taken directly from:

Robotic-Plant-Phenotyping

The scan launch file is the entry point for the automated acquisition trajectory. Test it repeatedly in Gazebo before connecting it to the physical robot.

3 Phase 2 — Physical JetCobot

3.1 Step A: Start the Jetson

Power the JetCobot and boot the Jetson Orin Nano.

From the development PC:

```
ping <JETSON_IP>
ssh jetson@<JETSON_IP>
```

3.2 Step B: Source the JetCobot Workspace

On the Jetson:

```
source /opt/ros/jazzy/setup.bash
source ~/jetcobot_ws/install/setup.bash
```

Check the installed packages:

```
ros2 pkg list | grep jetcobot
```

3.3 Step C: Start the Physical Robot Driver

Start the **real JetCobot driver supplied with the installed Yahboom software package**.

Because the exact driver launch filename depends on the JetCobot software image/version, first inspect the installed package:

```
ros2 pkg executables | grep jetcobot
```

and:

```
ros2 launch <jetcobot_driver_package> <real_driver_launch_file>
```

Do not use a Gazebo driver for the physical robot.

3.4 Step D: Verify Joint Feedback

Check that the physical robot publishes its joint states:

```
ros2 topic list | grep joint
```

Then:

```
ros2 topic echo /joint_states
```

The reported joint values must correspond to the actual physical configuration.

3.5 Step E: Start MoveIt2

On the control PC:

```
source /opt/ros/jazzy/setup.bash
source ~/jetcobot_ws/install/setup.bash

ros2 launch jetcobot_moveit moveit_rviz.launch.py
```

Verify that RViz reflects the physical robot state.

3.6 Step F: Execute One Simple Motion

Before starting the plant scan:

1. Select the `arm` planning group.
2. Choose a safe target pose.
3. Click **Plan**.
4. Check the complete trajectory.
5. Click **Execute**.

3.7 Step G: Run the Project Scan

After the physical driver and MoveIt2 have been verified, run the same **scan launch file used in simulation**, with its hardware/physical configuration enabled according to the parameters defined in the project.

```
ros2 launch plant_acquisition <scan_launch_file> <physical_parameters>
```

The exact command should be taken from the project's launch file rather than manually recreating the scan procedure.

Project repository:

Robotic-Plant-Phenotyping

3.8 Step H: Monitor the Robot

During the physical scan, monitor the ROS 2 system:

```
ros2 node list
ros2 topic list
ros2 topic echo /joint_states
```

Stop immediately if the robot behaves unexpectedly.

4 Phase 3 — RGB Acquisition

Once the physical scan is stable, integrate the wrist-mounted RGB camera.

1. Start the camera driver and verify the image topic.
2. Verify the camera calibration and image resolution.
3. Record image timestamps together with the robot pose.
4. Use the TF tree to associate each image with the camera pose.
5. Save the synchronized image–pose dataset.

The resulting dataset can then be processed by the reconstruction pipeline, including COLMAP and subsequent 3D reconstruction methods.

5 Quick Verification Checklist

☐ Ubuntu 24.04 installed on the development PC.

☐ ROS 2 Jazzy installed and sourced.

☐ Jetson Orin Nano configured.

☐ JetCobot workspace built successfully.

☐ MoveIt2 + RViz tested.

☐ Gazebo + plant world tested.

☐ Project scan launch tested in simulation.

☐ Physical JetCobot driver verified.

☐ Physical joint states verified.

☐ One simple physical MoveIt2 trajectory executed.

☐ Full scan executed on hardware.